Design, Validation and Verification document

1.0 Title of project and team members

Project Title: APEX (AI Patient Experience Simulator)
Team Members:

1. Tung Ho - 400370087
2. Asher Haroon - 400411801
3. Michael Fedotov - 400384934
4. Hammad Ur Rehman - 400370677
5. Christian Canlas - 400386605
6. Aaryan Kandwal - 400368334
7. Samir Matani - 400371096

2.0 Revision History

Version Number	Authors	Description	Date
0	● Tung Ho ● Asher Haroon ● Michael Fedotov ● Hammad Ur Rehman ● Christian Canlas ● Aaryan Kandwal ● Samir Matani	First Draft	January 14th, 2026

3.0 Introduction

3.1 Project Purpose

Effective clinical communication and empathy are critical skills for medical professionals, particularly when delivering difficult news to patients. However, opportunities for structured, repeatable, and scalable empathy training are limited due to resource constraints and variability in human role-play scenarios.

APEX (AI Patient Experience Simulator) is an AI-powered empathy training platform designed to allow medical trainees to practice patient-centered communication through simulated clinical conversations. The system leverages large language models (LLMs) to act as virtual patients and provides structured post-session feedback based on the SPIKES protocol. This enables trainees to receive immediate, consistent, and repeatable feedback on their empathetic communication skills.

3.2 Document Purpose

The purpose of this document is to provide a detailed description of the design and architecture of the APEX system. It outlines the system components, their interactions, and their relationship to the project requirements as defined in the Software Requirements Specification (SRS). This document also identifies potential undesired behaviours for each component to support future verification and validation efforts.

3.3 Project Scope

The APEX system consists of a web-based application that allows users to:

- Authenticate and manage training sessions
- Select predefined clinical cases
- Engage in text-based (and optional voice-based) conversations with simulated patients
- Receive structured feedback aligned with the SPIKES protocol
- View session transcripts and empathy-related metrics

The system is intended for training and educational purposes only and does not provide medical advice or diagnostic recommendations.

3.4 Assumptions and Invariants
3.4.1 Assumptions

- Users are medical trainees or instructors with basic clinical communication knowledge
- Users have access to a stable internet connection
- AI-generated responses are probabilistic and may vary between sessions
- The system is used in a controlled educational setting

3.4.2 Invariants

- The system does not replace real clinical training or supervision
- No real patient data is used within the system
- All final clinical judgments remain the responsibility of human practitioners
- Stored session data remains immutable once a session is closed

---

4.0 Project Component Diagram (High-Level)

The project component diagram is illustrated in Appendix A, Figure A.5.

The APEX system is composed of interconnected frontend, backend, and external components that support user interaction, session management, AI-driven dialogue, and analytics.

1. The Frontend UI sends user actions and session requests to the Backend.
2. The Backend returns processed responses and session updates to the Frontend UI.

3. The Backend sends prompts and contextual data to External LLM Services for AI-driven dialogue and analysis.
4. External LLM Services return generated responses and structured outputs to the Backend.
5. The Backend stores session data, dialogue turns, and feedback in the Database.

4.1 Backend Component Diagram

The project component diagram is illustrated in Appendix A, Figure A.6.

1. The frontend sends REST or WebSocket requests to the backend's API controllers.
2. The API controllers route each request to the appropriate backend service based on its purpose, such as authentication, case retrieval, session management, or dialogue processing.
3. For conversational input, the dialogue service loads and updates session context and protocol state as needed.
4. The dialogue service performs local natural language understanding by running internal rule-based analysis and span detection on the user's input.
5. When a simulated patient response is required, the dialogue service invokes external language model services through the backend's adapter layer and receives the generated output.
6. After processing a turn, the dialogue service calls the scoring service to compute empathy metrics, question classifications, and other evaluation signals.
7. The scoring service may use both internal language analysis and external language model services to produce structured feedback.
8. Conversation turns, session updates, and computed feedback are persisted through the data repositories to the database.
9. Once processing is complete, the API controllers return dialogue responses, session updates, and feedback results back to the frontend.
10. If voice interaction is enabled, the dialogue service may optionally invoke speech processing adapters before continuing the same processing flow.

5.0 Relationship Between Project Components and Requirements

System Component	Requirements Covered
Frontend UI	P0: Interactive training interface, session control Nonfunctional: Usability, accessibility
Backend API	P0: Session persistence and authentication P1: Role-based access P0: Simulated patient interaction

	P1: SPIKES stage detection P0: Empathy feedback generation
Database	P0: Transcript and feedback storage Nonfunctional: Reliability
External Services	P0: Reliable patient response generation for simulated dialogue based on case context and session state

---

6.0 Project Components

6.1 Frontend UI

6.1.1 Normal Behaviour

The UI allows users to log in, select a clinical case, initiate a training session, and interact with a simulated patient through a chat interface. Upon completion of a session, users can view structured feedback and session summaries.

6.1.2 API and Structure

The UI communicates with the backend via RESTful API endpoints to:

- Authenticate users
- Create and manage sessions
- Submit dialogue turns
- Retrieve feedback and session summaries

Data is exchanged in JSON format.

6.1.3 Implementation

The UI is implemented using React with TypeScript. State management is handled through a centralized store, and routing is managed via a client-side router. The chat interface renders conversation turns dynamically and updates in real time as responses are received from the backend.

6.1.4 Potential Undesired Behaviours

- User input submission failures due to network interruptions
- UI latency when backend responses are delayed
- Incomplete rendering of session feedback
- Unauthorized access if authentication tokens are mismanaged

6.2 Backend

6.2.3 API LAYER (Controllers)

Cases Controller

a. Normal behaviour: provides CRUD operations for patient simulation cases with role-based access control.

b. API (inputs/outputs): inputs include case metadata (title, script, objectives, difficulty, category) and query parameters for filtering and pagination. Outputs include case objects and paginated lists of cases.

c. Implementation

- Endpoints:
    - `GET /cases, POST /cases, GET /cases/{id}, PATCH /cases/{id}, and DELETE /cases/{id}`
- An instructor role is required to create/update/delete
- Delegates logic to `CaseService`

d. Potential undesired behaviours: missing input sanitization, no validation on difficulty/category enums, and no content length limits.

Sessions Controller

a. Normal behaviour: manages session creation, turn processing (text/audio), and session closure.

b. API (inputs/outputs)

- Inputs: session creation parameters (case ID), text turns, and audio files.
- Outputs: session state, patient responses, and feedback metrics.

c. Implementation

- Endpoints:
    - `POST /sessions, POST /sessions/{id}/turns, POST /sessions/{id}/audio, and POST /sessions/{id}/close`
- Instantiates adapters (LLM, Whisper, NLU)
- Orchestrates `DialogueService → ScoringService`

d. Potential undesired behaviours

- Adapter instantiation per request (performance overhead)
- No audio size or format validation
- Race conditions on concurrent turn submissions

6.3.2 BUSINESS LOGIC LAYER (Services)

Case Service

a. Normal behaviour: manages creation, retrieval, updating, and deletion of cases.

b. API (inputs/outputs)

- Inputs: case metadata, filters, and pagination parameters.
- Outputs: case entities and lists.

c. Implementation

- Uses `CaseRepository`
- Validates case existence
- Partial updates via `model_dump(exclude_unset=True)`

d. Potential undesired behaviours: duplicate cases not detected, and enum values not validated.

Session Service

a. Normal behaviour: controls session lifecycle and session metadata.

b. API (inputs/outputs): inputs include Case ID, user ID, session ID. Outputs include session entity with turn history.

c. Implementation

- Initializes sessions at SPIKES stage “Setting”
- Calculates session duration on closure
- Retrieves turns via `TurnRepository`

d. Potential undesired behaviours: missing permission checks, unlimited concurrent sessions, and orphaned sessions remain open.

Dialogue Service (SPIKES Engine)

a. Normal behaviour: processes dialogue turns and advances SPIKES stages.

b. API (inputs/outputs): inputs include session ID, clinician turn text. Outputs include patient response turn, and metrics.

c. Implementation

- Pipeline: input → NLU → turn creation → LLM → EO detection → SPIKES update
- NLU via `SimpleRuleNLU`

- Span detection with character offsets
- LLM generates patient responses
- SPIKES stage progression via keyword and fallback logic

d. Potential undesired behaviours: no validation of LLM output length/content

Scoring Service

a. Normal behaviour: generates empathy and SPIKES feedback using AFCE metrics.

b. API (inputs/outputs): inputs include the session ID. Outputs include scores, metrics, and structured feedback.

c. Implementation

- Empathy score (EO coverage, dimension match, timing)
- SPIKES score (coverage, sequence, empathy during E)
- Links EOs to responses within two turns
- Computes missed opportunities

d. Potential undesired behaviours: serialization errors with special characters and no caching (recalculation on every call)

6.3.3 NLU COMPONENTS

Rule-Based NLU (SimpleRuleNLU)

a. Normal behaviour: detects empathy cues, question types, tone, and AFCE constructs.

b. API (inputs/outputs)

- Inputs: text string
- Outputs: classified cues, tone, spans

c. Implementation: keyword-based empathy detection, question classification (open vs closed), tone analysis, and delegates span extraction to `SpanDetecto.`

d. Potential undesired behaviours: no negation handling and English-only support

Span Detector

a. Normal behaviour: identifies AFCE and SPIKES spans with offsets.

b. API (inputs/outputs)

- Inputs: text string
- Outputs: span objects (start, end, confidence, type)

c. Implementation

- EO detection across dimensions and types, elicitation and response detection, SPIKES stage keyword matching, and overlap resolution by confidence.

d. Potential undesired behaviours

- Fragmented multi-word spans, imperfect overlap resolution, no contextual understanding, and arbitrary confidence scores.

6.3.4 Data Access Layer (Repositories)

Repositories (User, Case, Session, Turn, Feedback)

a. Normal behaviour: provide CRUD access to persistent storage.

b. API (inputs/outputs): inputs include ORM entities and filters. Outputs include Database records.

c. Implementation (Methods): `get_by_id`, `get_all`, `create`, `update`, `delete,` SQLAlchemy ORM, and pagination via `offset` and `limit`

d. Potential undesired behaviours: no query timeouts, no soft deletes, and incomplete transaction rollback

Speech Adapter (Whisper)

a. Normal behaviour: Transcribes clinician audio input to text.

b. API (inputs/outputs): inputs include audio bytes (wav/mp3/ogg). Outputs include transcribed text.

c. Implementation: writes audio to a temporary file and calls Whisper API externally.

d. Potential undesired behaviours: missing audio validation and temporary files not deleted on failure.

6.5 Database

6.5.1 Normal Behaviour

The database provides persistent storage for the APEX system, including users, synthetic patient cases, dialogue sessions, turn-level interactions, and automated feedback. Session data is written incrementally during conversations and finalized upon completion, supporting both real-time interaction and post-session analysis.

6.5.2 API and Structure

APEX uses a normalized PostgreSQL schema with foreign key relationships to maintain referential integrity across users, sessions, turns, and feedback records. Analytical outputs, including empathy metrics and SPIKES annotations, are stored in serialized form for flexibility. Database access is managed through the backend API with role-based access control and restricted write permissions.

6.5.3 Implementation

The database is implemented using PostgreSQL and hosted on Supabase. Application access is managed through SQLAlchemy ORM, providing transactional guarantees and automatic rollback on failure. Core tables are indexed to support efficient session retrieval and analytics.

6.5.4 Potential Undesired Behaviors

- Incomplete or corrupted writes, where sessions, turns, or feedback records are only partially persisted due to interrupted database transactions
- Unauthorized or unintended data access, caused by misconfigured authentication, role-based access control, or API permissions
- Performance degradation under load, particularly during high-frequency turn logging or analytics-heavy queries

---

7.0 User Interface

The APEX frontend consists of multiple views, including:

- Login screen, case selection dashboard, chat-based training interface, and Feedback summary page

The interface prioritizes clarity, minimal cognitive load, and ease of navigation to support effective learning. Visual indicators are used to display session progress and SPIKES stages.

The chat-based training interface optionally supports voice-based interaction through microphone input and audio playback, with automatic fallback to text-based interaction if audio is unavailable.

Screenshots which represent the APEX user interface, illustrating the login flow, case selection, chat-based training interface, and post-session feedback view, are provided in Appendix B. These screenshots illustrate the flow of the user interface (UI) that matches the V0 implementation that is described in this document.

8.0 Validation and Verification

This section describes the validation and verification activities used to ensure that the system meets its functional and nonfunctional requirements. Verification focuses on confirming the correct behavior of individual components, while validation evaluates performance, robustness, and usability under realistic operating conditions.

8.1 User Interface

8.1.1 Unit Tests

- Verify successful login with valid credentials
- Verify rejection of invalid or missing credentials with appropriate error messages
- Verify correct case selection and navigation between views
- Verify correct submission and display of chat messages
- Verify correct rendering of feedback summary information

8.1.2 Performance Tests and Metrics

- Measure page load time for main views
- Observe interaction latency during normal user workflows
- Interface is considered acceptable if interactions remain responsive during typical usage

8.2 Backend API

8.2.1 Unit Tests

- Verify authenticated access to session lifecycle endpoints
- Verify rejection of unauthenticated or unauthorized requests
- Verify correct request and response handling for dialogue turn submission
- Verify successful session closure triggers downstream scoring logic
- Verify correct error responses for malformed or invalid API requests
- Verify feedback retrieval is rejected for sessions that are not yet completed

8.2.2 Performance Tests and Metrics

- Measure response latency for core API endpoints
- Backend responses are expected within a time frame suitable for interactive web applications

8.3 Dialogue Service

8.3.1 Unit Tests

- Verify that a single trainee submission results in:
    - Creation and persistence of one trainee (user) turn
    - Creation and persistence of one virtual patient (assistant) turn

- Verify correct orchestration of dialogue processing across turns
- Verify accurate extraction and serialization of:
    - Elicitation spans
    - Response spans
    - Empathic opportunity (EO) spans
- Verify correct assignment and progression of SPIKES stages at the turn and session level
- Verify correct handling of empty, malformed, or unexpected user input
- Verify that no scoring or feedback generation is triggered during active dialogue processing

8.3.2 Performance Tests and Metrics

- Measure time from user input submission to response generation
- Dialogue response latency is expected to support real time interaction

8.4 Scoring Service

8.4.1 Unit Tests

- Verify that the scoring service is invoked only upon session closure
- Verify that scoring is not performed during active dialogue turns
- Verify correct construction of the scoring input payload from:
    - Ordered session transcript
    - Turn roles and metadata
    - Available annotations
- Verify correct handling of evaluator outputs, including:
    - Valid structured outputs
    - Invalid, incomplete, or non-conforming outputs
- Verify enforcement of score bounds for all quantitative scores
- Verify persistence of exactly one feedback record per completed session
- Verify explicit failure behavior for scoring errors rather than silent or partial feedback generation

8.4.2 Performance Tests and Metrics

- Measure the time required to compute feedback scores
- Scoring is expected to be completed without introducing noticeable delay

8.5 Database

8.5.1 Unit Tests

- Verify correct creation and retrieval of: session records, turn records and feedback records.
- Verify correct persistence of: turn-level annotations, and session metadata.
- Verify that feedback records are uniquely associated with completed sessions

- Verify correct update behavior for existing feedback records
- Verify system behavior under partial failure scenarios where:
    - Earlier database commits succeed
    - Subsequent related writes fail
- Verify resulting database state matches documented persistence behavior

8.5.2 Performance Tests and Metrics

- Measure query response times for common database operations
- Database performance is expected to remain stable under session level usage

8.6 Audio Processing Module

8.6.1 Unit Tests

- Verify audio input is accepted when microphone permissions are granted
- Verify rejection or fallback behavior when microphone permissions are denied
- Verify that automatic speech recognition produces a valid transcript from audio input
- Verify ASR transcripts are processed identically to typed user input
- Verify that text-to-speech generates playable audio output for system responses
- Verify handling of audio upload interruptions and malformed audio input
- Verify ASR and TTS failures trigger automatic fallback to text-based interaction
- Verify audio-related metadata is correctly associated with the corresponding dialogue turn

8.6.2 Performance Tests and Metrics

- Measure end-to-end latency from audio submission to transcript availability
- Measure latency from text submission to audio playback availability
- Audio interaction is considered acceptable if the latency supports interactive training use
- For future extensions, paralinguistic feature outputs are sanity checked for valid ranges
- Paralinguistic analysis results are treated as advisory only and do not affect core system flow

8.7 Summary

The validation and verification activities described above provide confidence that system components behave correctly, meet performance expectations, and degrade safely under failure conditions. Quantitative metric values and test results will be reported in the final submission.

9.0 Appendix A

9.1 Database

The application uses a PostgreSQL database to store information related to users, synthetic patient cases, dialogue sessions, turn-level conversation logs, and generated feedback. The core tables and their DDL are provided below:

9.1.1 Cases Table

Column Name	Data Type	Description	Constraint
id	INTEGER	A unique identifier for each patient case	PRIMARY KEY
title	VARCHAR	The title of the patient case	NOT NULL
description	TEXT	A brief description of the case scenario	NOT NULL
script	TEXT	The base script or narrative used to guide the simulated patient responses	NOT NULL
objectives	TEXT	Learning objectives associated with the case	NOT NULL
difficulty_level	VARCHAR	The difficulty level of the case (e.g., easy, medium, hard)	NOT NULL
category	VARCHAR	The category or type of patient scenario	NOT NULL
patient_background	TEXT	Background information about the synthetic patient	NOT NULL
expected_spikes_flow	TEXT	The expected progression of the case aligned with SPIKES stages	NOT NULL
created_at	TIMESTAMP	Timestamp indicating when the case was created	NOT NULL

updated_at	TIMESTAMP	Timestamp indicating when the case was last updated	NOT NULL

9.1.2 Users Table:

Column Name	Data Type	Description	Constraint
id	INTEGER	A unique identifier for each user	PRIMARY KEY
email	VARCHAR	The email address associated with the user account	UNIQUE, NOT NULL
hashed_password	VARCHAR	Securely hashed password for authentication	NOT NULL
role	VARCHAR	Role of the user (e.g., trainee, admin)	NOT NULL
full_name	VARCHAR	Full name of the user	NOT NULL
created_at	TIMESTAMP	Timestamp indicating when the user account was created	NOT NULL
updated_at	TIMESTAMP	Timestamp indicating when the user account was last updated	NOT NULL

9.1.3 Turns Table:

Column Name	Data Type	Description	Constraint
id	INTEGER	A unique identifier for each conversation turn	PRIMARY KEY
session_id	INTEGER	Identifier of the conversation session this turn belongs to	FOREIGN KEY
user_id	INTEGER	Identifier of the user who produced the turn	FOREIGN KEY

turn_number	INTEGER	Sequential order of the turn within a session	NOT NULL
role	VARCHAR	Role of the speaker (e.g., learner, patient)	NOT NULL
text	TEXT	Text content of the turn	NOT NULL
audio_url	VARCHAR	Optional URL linking to the audio recording of the turn	NULL
metrics_json	TEXT	Serialized metrics related to empathy and communication analysis	NULL
spans_json	TEXT	Serialized span-level annotations for the turn	NULL
relations_json	TEXT	Serialized relationships between annotated spans	NULL
spikes_stage	VARCHAR	SPIKES stage associated with this turn	NULL
timestamp	TIMESTAMP	Timestamp indicating when the turn occurred	NOT NULL

9.1.4 Sessions Table:

Column Name	Data Type	Description	Constraint
id	INTEGER	A unique identifier for each conversation session	PRIMARY KEY
user_id	INTEGER	Identifier of the user who initiated the session	FOREIGN KEY
case_id	INTEGER	Identifier of the patient case used in the session	FOREIGN KEY
state	VARCHAR	Current lifecycle state of the session (e.g., active, completed)	NOT NULL
current_spikes_stage	VARCHAR	Current SPIKES stage of the session	NULL

started_at	TIMESTAMP	Timestamp indicating when the session started	NOT NULL
ended_at	TIMESTAMP	Timestamp indicating when the session ended	NULL
duration_seconds	INTEGER	Total duration of the session in seconds	NULL
metadata	TEXT	Additional session-level metadata stored in serialized form	NULL

9.1.5 Feedback Table:

Column Name	Data Type	Description	Constraint
id	INTEGER	A unique identifier for each feedback record	PRIMARY KEY
session_id	INTEGER	Identifier of the session this feedback corresponds to	FOREIGN KEY
empathy_score	DOUBLE PRECISION	Quantitative score representing empathetic communication	NOT NULL
communication_score	DOUBLE PRECISION	Score reflecting overall communication effectiveness	NOT NULL
spikes_completion_score	DOUBLE PRECISION	Score indicating completion of SPIKES stages	NOT NULL
overall_score	DOUBLE PRECISION	Aggregate performance score for the session	NOT NULL
eo_counts_by_dimension	TEXT	Counts of empathic opportunities grouped by empathy dimension	NULL

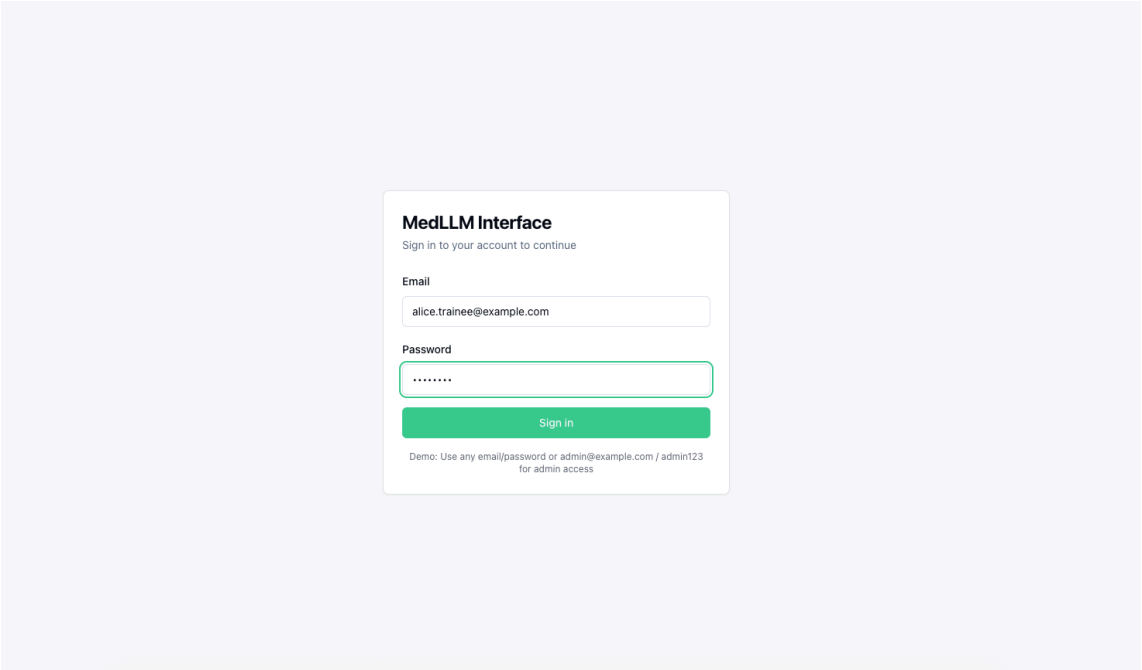
elicitation_counts_by_type	TEXT	Counts of elicitation strategies used by type	NULL
response_counts_by_type	TEXT	Counts of empathic responses grouped by response type	NULL
eo_counts	TEXT	Total counts of empathic opportunities identified	NULL
elicitation_count	INTEGER	Total number of elicitation attempts	NULL
empathy_response_count	INTEGER	Total number of empathic responses	NULL
linkage_stats	TEXT	Statistics describing linkages between EO, elicitation, and responses	NULL
missed_opportunities	TEXT	Identified missed empathic opportunities	NULL
missed_opportunities_by_dimension	TEXT	Missed opportunities grouped by empathy dimension	NULL
eo_to_elicitation_links	TEXT	Mappings between empathic opportunities and elicitation attempts	NULL
eo_to_response_links	TEXT	Mappings between empathic opportunities and responses	NULL
response_types	TEXT	Distribution of response types used by the trainee	NULL
spikes_coverage	TEXT	Coverage information for SPIKES stages	NULL

spikes_timestamps	TEXT	Timestamps associated with SPIKES stage transitions	NULL
spikes_strategies	TEXT	Strategies used by the trainee for each SPIKES stage	NULL
question_breakdown	TEXT	Breakdown of question types used during the session	NULL
bias_probe_info	TEXT	Metadata related to bias probe analysis, if applicable	NULL
evaluator_meta	TEXT	Metadata about the evaluation process or model configuration	NULL
latency_ms_avg	DOUBLE PRECISION	Average model response latency in milliseconds	NULL
strengths	TEXT	Identified communication strengths	NOT NULL
areas_for_improvement	TEXT	Suggested areas for improvement	NOT NULL
detailed_feedback	TEXT	Detailed narrative feedback for the session	NOT NULL
created_at	TIMESTAMP	Timestamp indicating when the feedback was generated	NOT NULL

9.2 User Interface Flow and Screenshots

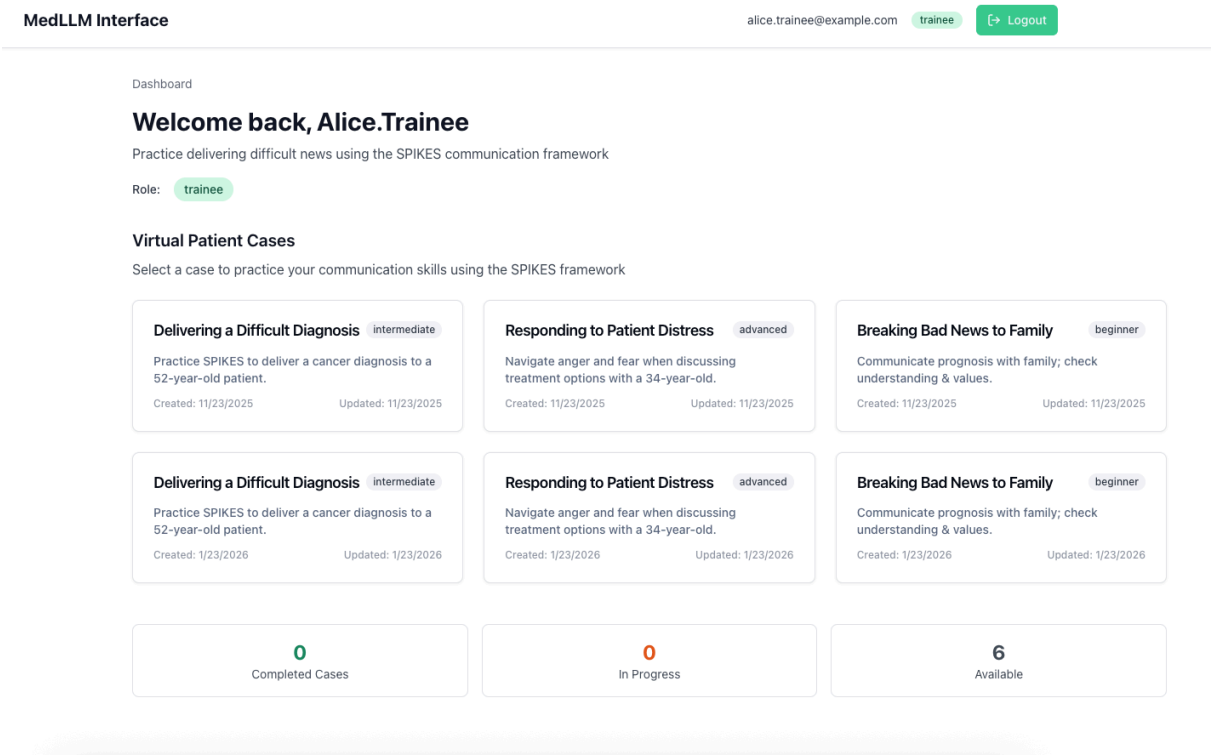
9.2.1 Login and Authentication Screen

Figure A.1 shows the login interface used to authenticate users before accessing the APEX system. Valid credentials are needed to proceed to the case selection dashboard. Authentication is implemented and enforced by the backend API. If invalid credentials are provided, they are rejected with an error message



9.2.2 Case Selection Dashboard

Figure A.2 shows the case selection dashboard, where users choose from predefined “Virtual Patient Cases”. A user selects a case by clicking a case tile and is redirected to the chat interface for that case, initializing a new training session. Each case displays relevant metadata such as title, difficulty level, created/updated dates and learning objectives.



9.2.3 Chat-Based Training Interface

Figure A.3a shows the clinical case script and contextual information presented to the LLM for the active training session. The view defines the case metadata, and the script defines the virtual patient's background, emotional state, clinical context, and behavioural rules. As well, following the SPIKES format, the expected stages are outlined for this scenario. The script is used to create a flow for speech and conversation with the virtual simulated patient. This primarily ensures that responses from the dialogue remain consistent with the intended learning objectives.

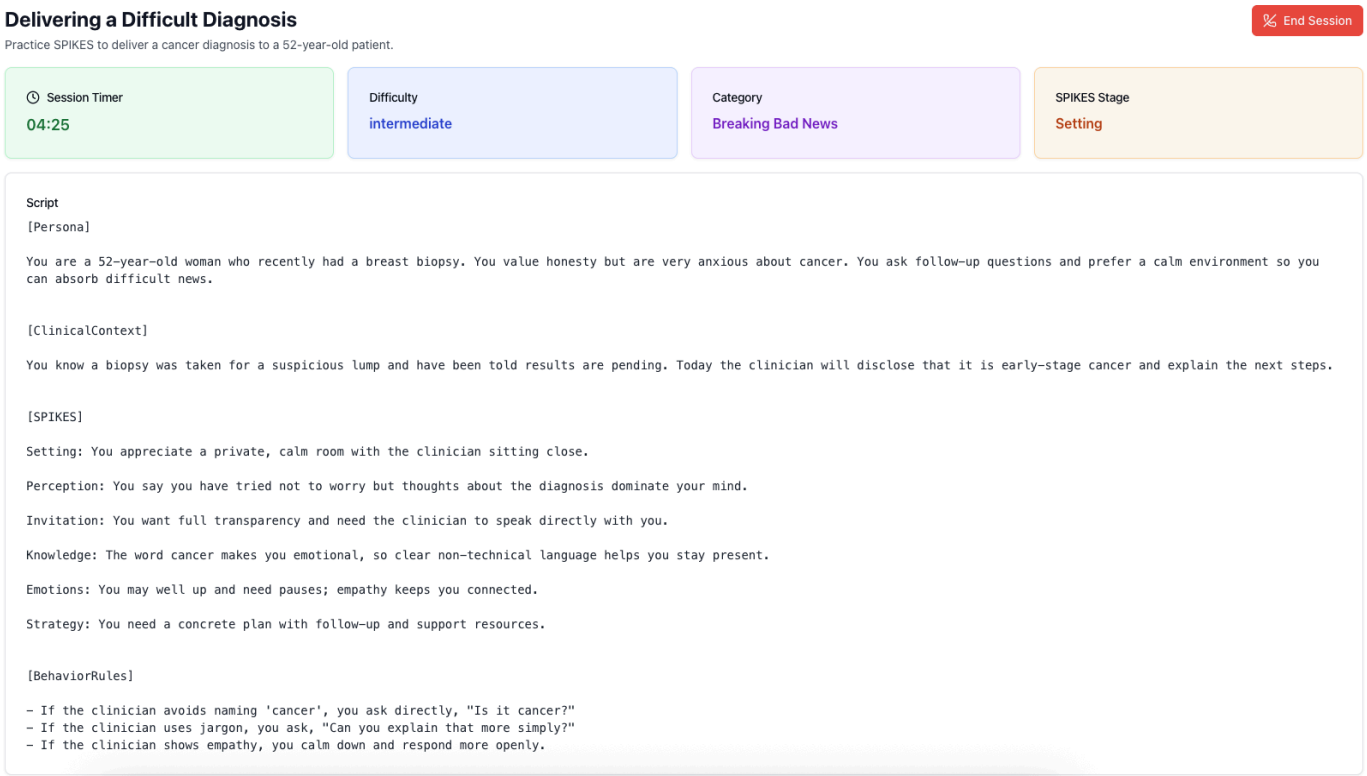
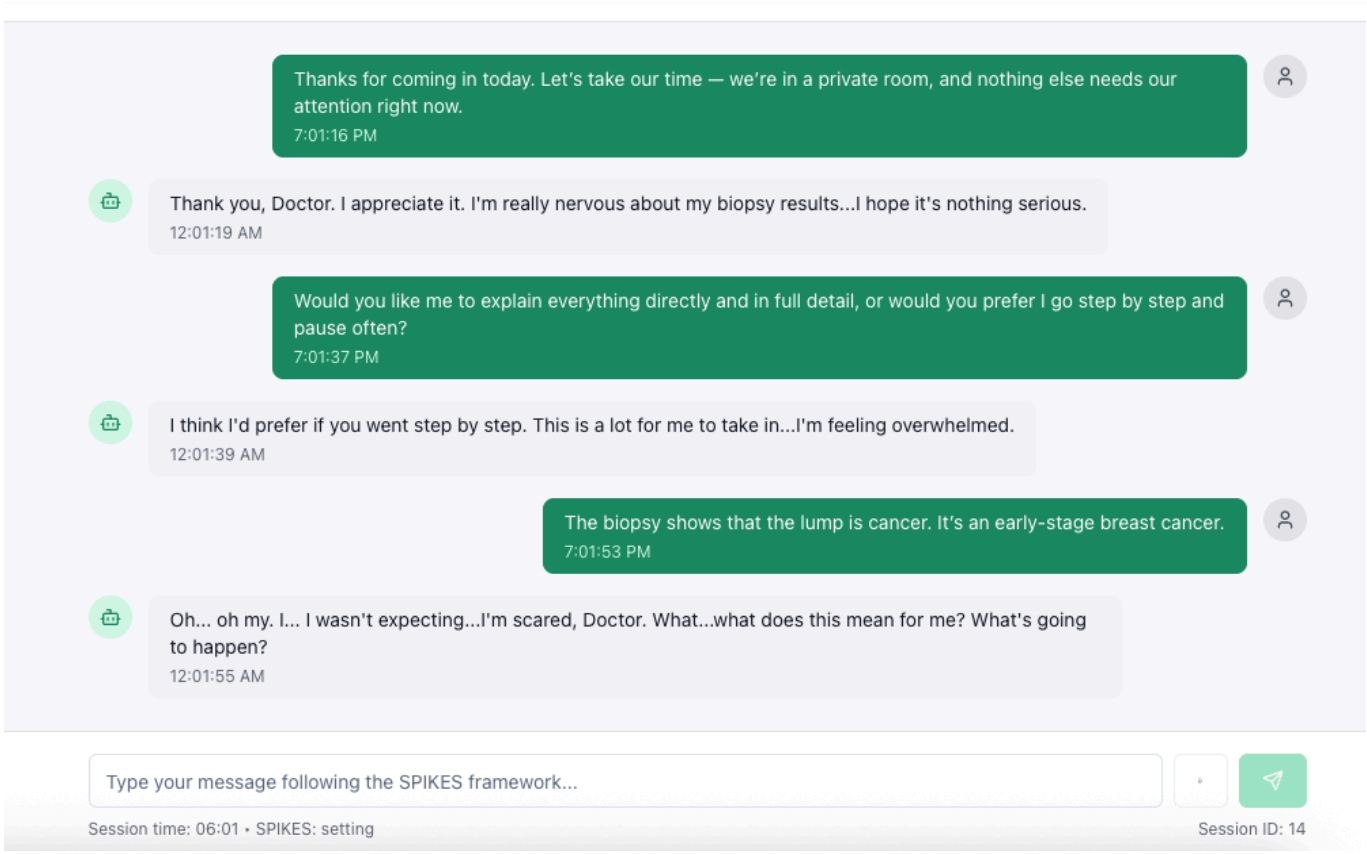
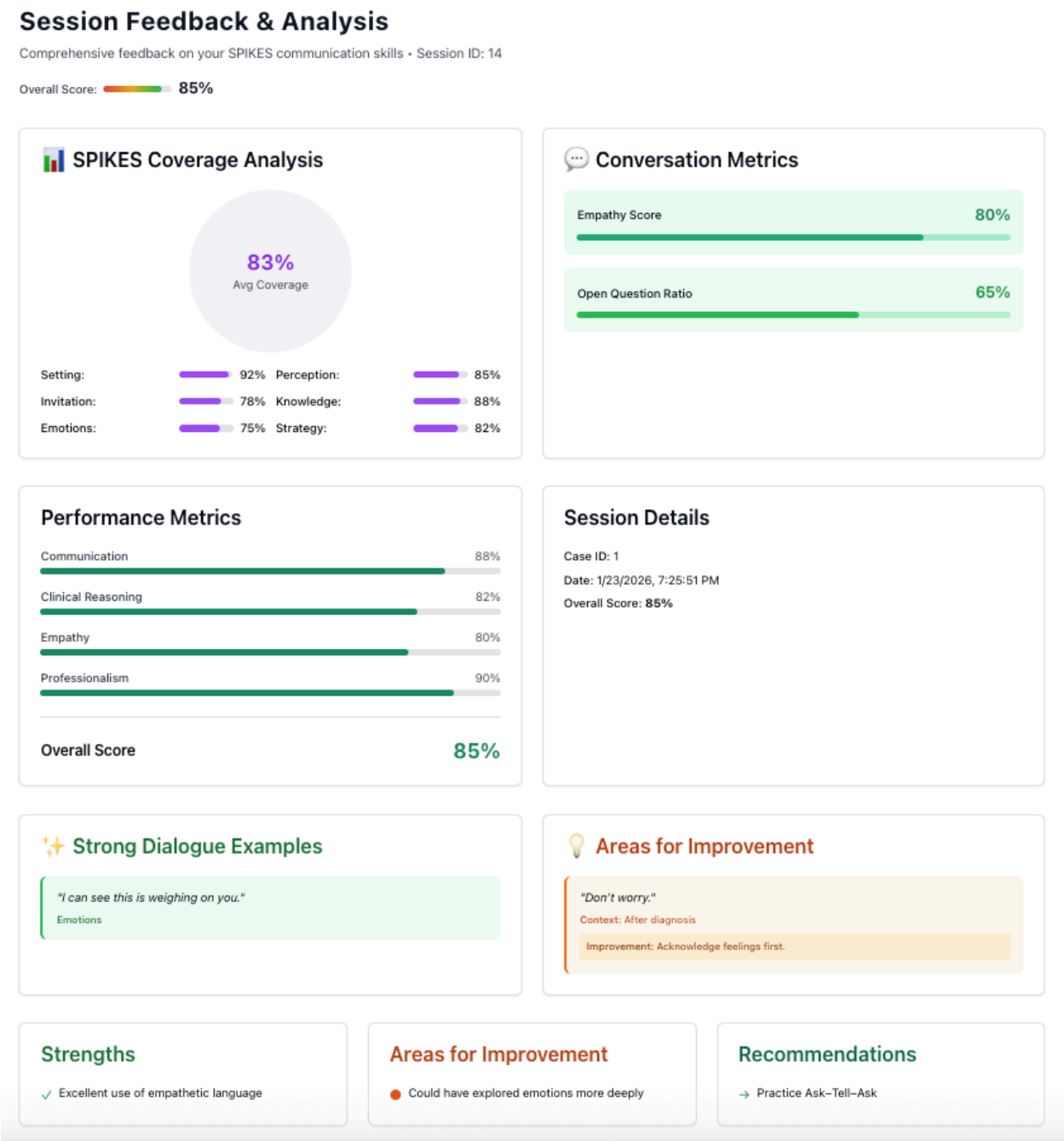


Figure A.3b shows the chat conversation with the virtual patient. The chat input is used to accept dialogue that is then processed by the backend and the external LLM provider. The virtual patient's response is then displayed in the chat.



9.2.4 Session Feedback and Scoring View

Figure A.4 shows the feedback and analysis screen after ending the conversation with the virtual patients. SPIKES coverage analysis provides percentage metrics for each SPIKES component reached. Some additional details regarding the conversation are provided, such as: strengths, areas for improvement, recommendations, strong dialogue examples and conversation metrics.



9.3

Figure A.5, Project Component Diagram

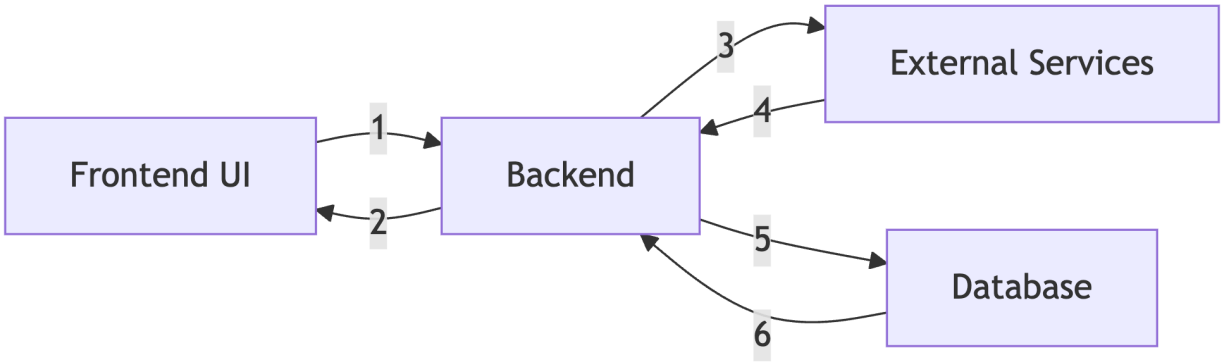
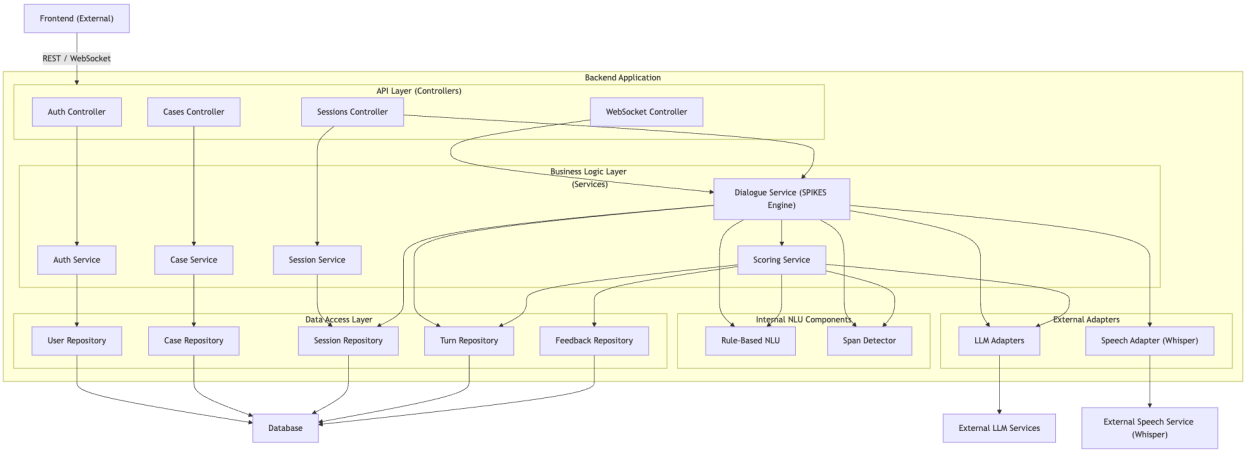



Figure A.6 Backend Component Diagram